RESEARCH-ARTICLE

Identifying Efficient Dataflows for Spiking Neural Networks

DEEPIKA SHARMA, Purdue University, West Lafayette, IN, United States

AAYUSH ANKIT, Microsoft Corporation, Redmond, WA, United States

KAUSHIK ROY, Purdue University, West Lafayette, IN, United States

Open Access Support provided by:

Purdue University

Microsoft Corporation

Published: 01 August 2022

ISLPED '22: ACM/IEEE International Symposium on Low Power Electronics and Design
August 1 - 3, 2022
MA, Boston, USA

Conference Sponsors:
SIGDA

Identifying Efficient Dataflows for Spiking Neural Networks

Deepika Sharma
Purdue University
West Lafayette, Indiana, USA
sharm444@purdue.edu

Aayush Ankit*
Microsoft Corporation
Mountain View, California, USA
aayushankit@microsoft.com

Kaushik Roy
Purdue University
West Lafayette, Indiana, USA
kaushik@purdue.edu

ABSTRACT

Deep feed-forward Spiking Neural Networks (SNNs) trained using appropriate learning algorithms have been shown to match the performance of state-of-the-art Artificial Neural Networks (ANNs). The inputs to an SNN layer are 1-bit spikes distributed over several timesteps. In addition, along with the standard artificial neural network (ANN) data structures, SNNs require one additional data structure – the membrane potential (Vmem) for each neuron which is updated every timestep. Hence, the dataflow requirements for energy-efficient hardware implementation of SNNs can be different from the standard ANNs. In this paper, we propose optimal dataflows for deep spiking neural network layers. To evaluate the energy and latency of different dataflows, we considered three hardware architectures with varying on-chip resources to represent a class of spatial accelerators. We developed a set of rules leading to optimum dataflow for SNNs that achieve more than 90% improvement in Energy-Delay Product (EDP) compared to the baseline for some workloads and architectures.

CCS CONCEPTS

• **Computer systems organization** → **Neural networks**; *Special purpose systems*; • **General and reference** → Experimentation.

KEYWORDS

deep spiking neural networks, artificial neural networks, dataflows

ACM Reference Format:
Deepika Sharma, Aayush Ankit, and Kaushik Roy. 2022. Identifying Efficient Dataflows for Spiking Neural Networks. In *ACM/IEEE International Symposium on Low Power Electronics and Design (ISLPED '22), August 1–3, 2022, Boston, MA, USA.* ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3531437.3539704

1 INTRODUCTION

Inspired by biological neurons with leaky-integrate-and-fire (LIF) activation and computing with spikes, research on Spiking Neural Networks (SNNs) has started in earnest as an energy-efficient alternative to standard artificial neural networks (henceforth, denoted as ANNs). SNNs, unlike ANNs, can be effective in processing sequential information since it's membrane potentials store information of past inputs. In other words, SNNs can potentially mimic Recurrent Neural Networks (RNNs). In the recent past, several research efforts have been expended on training deep feed-forward spiking neural networks for challenging workloads [4, 12, 18, 20, 21, 24]. In this work, we focus on such SNNs suitable for complex tasks.

ANNs have been able to achieve high accuracy for various deep learning tasks, albeit at a high energy cost[15]. The fundamental computation in ANNs is the expensive multiplication and accumulation (MAC) operation and the corresponding access, movement, write-back of data from the memory and the processing units. Whereas, SNNs work with sparse binary spikes distributed over several timesteps and have accumulate (AC) as the fundamental operation[3]. However, data movements from memory and processing units are still required. In addition, SNNs update the membrane potential every timestep.

With the growing size of deep-learning models, it is not possible to store all the data on-chip. Thus, storing and accessing off-chip data consume a significant amount of energy and can be the bottleneck for ANN and SNN implementations. To reduce off-chip communication, researchers have proposed several dataflows for ANNs that try to decrease the data movement by increasing data reuse. Authors in [6] proposed weight, output and input stationary (WS, OS and IS) dataflow taxonomy for ANNs depending on the most reused data structure. In addition, they introduced a novel row stationary (RS) dataflow that produces row-wise outputs and maximizes the reuse of all data structures.

To reduce the data movement between off-chip DRAM and on-chip memory in SNN hardware implementations, we analyzed different dataflow variants for SNNs starting with an ANN-like baseline for three spatial hardware architectures detailed in Sections 3.1 and 4. One way to convert an ANN dataflow to its equivalent SNN dataflow is to perform all the computations for one timestep before moving to the next timestep, which we consider as the baseline SNN dataflow. We developed a set of directives to create efficient SNN dataflows from these baselines.

The contributions of this work are summarized as follows:

- We establish the need for SNN dataflow exploration by showing performance dependence on architecture and workload for deep feed-forward spiking networks.
- We develop a set of directives that lead to a dataflow that gives optimal performance (lower energy-delay-product, EDP) for an SNN architecture.
- We compare optimal row, output and weight stationary SNN dataflows for a variety of workloads.

2 BACKGROUND AND RELATED WORKS

2.1 Spiking Neuron: Background

In the last few decades, several spiking neuron models have been proposed [10]. Among these models, integrate-and-fire (IF) and

*Work done while affiliated with Purdue University

ISLPED '22, August 1–3, 2022, Boston, MA, USA

ACM ISBN 978-1-4503-9354-6/22/08.
https://doi.org/10.1145/3531437.3539704

leaky integrate-and-fire (LIF) neuron models have been widely used by recent learning algorithms [21] [12] [18]. An IF neuron has a membrane potential (V_{mem}) corresponding to each neuron. When an input spikes, the weight corresponding to that input is added to the neuron's V_{mem}. If V_{mem} exceeds the threshold (V_{th}) of the neuron, an output spike is generated, and the V_{mem} is reset to zero. An LIF neuron has an additional leak factor L corresponding to each neuron which allows the V_{mem} to decrease with timesteps. The dynamics of the IF and LIF neuron model's membrane potential, $V_{mem}(t)$, at timestep t is described by:

$$V_{mem}(t) = V_{mem}(t-1) + \sum_i (W_i * I_i(t)) - L \quad (1)$$

where $I_i(t)$ is the value of the i_{th} input to the neuron at timestep t, W_i is the corresponding weight for this input and L is a constant leak factor (zero in case of IF neuron).

2.2 SNN: Hardware Implications

A deep feed-forward SNN layer requires four data structures, namely inputs, weights, Vmems and outputs along with the threshold and leak parameters. For our discussion, we assume that the threshold and leak parameters are same for all the neurons in a layer. The weights corresponding to non-zero input spikes are accumulated over the input channels to obtain the Vmems (Fig. 1). The Vmems are then stored and updated every timestep to generate the output spikes.

Contemporary ANN and SNN accelerators have a multi-level storage hierarchy [5, 11, 19] including off-chip DRAM, on-chip global buffers, register files etc. A multi-level architecture keeps relevant data closer to compute and hides the access latency from lower storage level. There are multiple ways to schedule/map a workload on a hardware architecture (i.e. multiple dataflows) with orders of magnitude difference in their efficiencies [17]. Hence, there is a need to determine an optimal dataflow that effectively utilizes such an architecture.

Dataflow is defined by the partition/tiling of data structures among storage levels and the order of computations [17]. For any deep feed-forward spiking neural network layer, at each storage level, the order of computations and the required memory accesses can be defined by an eight-dimensional nested loop structure as shown in Fig. 1. The loop dimensions are T (timesteps), N (batch size), K and C (output and input channels, respectively), PxQ (output feature map dimension), RxS (weight kernel dimension), and WxH (input feature map dimension), where W=P+R-stride and H=Q+S-stride[9].

Each data structure has a subset of layer dimensions as indices, and every unique combination of the index values represents a unique point of that data structure. For the example in Fig. 1, the indices of weights are K, C, R and S, and every unique KCRS combination represents a weight value. The partition of a data structure stored at any level in a multi-level architecture is the product of its indices mapped to all the storage elements at the same or higher level as this element. For example, for a three-level architecture containing DRAM, global buffer and register files, the partition of weights stored on the global buffer is the product of K, C, R and S at the register file and global buffer level.

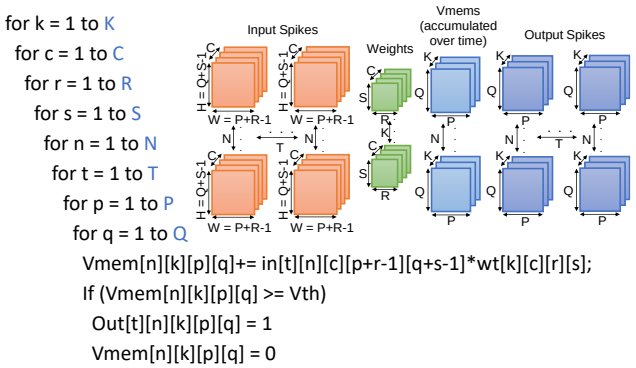

Figure 1: SNN as an eight-dimensional loop structure

For the given loop order KCRSNTPQ in Fig. 1, for every KCRS combination, the loop runs over all the other dimensions (NTPQ). It means that after fetching, a weight value is held stationary and reused for all the relevant computations across batch-size (N), timesteps (T) and output feature map dimension (PxQ) before fetching the next weight. Therefore, it represents a weight stationary (WS) dataflow. Utilizing such data reuse opportunities is crucial for optimizing dataflows.

Even though deep feed-forward SNNs are similar to ANNs, existing ANN dataflows cannot be used directly for these networks as the additional data structure (Vmem) and dimension timestep (T) make the computation and data reuse patterns in SNNs more complicated than ANNs. Several SNN dataflows can be developed from an ANN dataflow only by changing the position of the T dimension in the loop. As an example, at each storage level, there are 5040 (7 factorial) possible permutations for ANNs, whereas 40320 (8 factorial) permutations for SNNs. This is further compounded by changing the data partition among different storage levels. This growth in the dataflow search space gives more opportunities for optimization but, it also makes the search for optimal dataflow difficult, reinforcing the need for proper SNN dataflow exploration.

2.3 Related Works

In recent years, tremendous progress has been made in developing SNN training algorithms that achieve state of the art accuracy [12, 18, 21, 24]. In addition, several efficient learning algorithms are being developed for edge applications that use energy efficient sensors as inputs – one such application uses vision sensors (Dynamic Vision Sensors (DVS) provide output in the form of spikes, and hence are suitable for SNNs) to efficiently estimate optical flow, depth and detect objects in aerial robotics that require ultra low energy consumption [13] [22]. It is essential to consider energy-efficient hardware implementation of SNNs to support such applications.

Research on efficient hardware architecture development for SNNs has started in earnest. Both IBM and Intel have developed non-commercial hardware – TrueNorth[1] and Loihi[7]. These projects focus on emulating a brain-like structure by maximizing the number of neurons on hardware, allowing complex connectivity and event-driven computation. However, they do not discuss dataflow mapping in detail. Authors in [23] developed a dataflow-based

Table 1: SNN hardware research paradigms and contributions

Project	Novelty/Contribution
TrueNorth[1]	Hybrid asynchronous-synchronous circuit design flow and supporting tools
Loihi[7]	Asynchronous circuits and communication
SpinalFlow[16]	A new spike representation and a dataflow and architecture tailored for that
DFSyhthesizer[23]	Dataflow-based modelling of SNNs for neuromorphic hardware
SNN Dataflow for Systolic arrays[14]	Dataflow exploration framework for systolic array accelerators
This work	Dataflow exploration for deep spiking networks for 2-D spatial accelerators

framework for modeling SNN workloads on such crossbar based neuromorphic accelerators.

Similarly, there have been efforts to explore SNN dataflows. SpinalFlow [16] presents a dataflow and architecture tailored for a compressed, time-stamped, and sorted representation for temporally coded spikes. Nonetheless, a majority of SNN learning algorithms are based on rate coded spikes[2] [8]. Authors in [14] discuss the effects of changing the position of T and built a simulator for design-space exploration of SNN dataflows for systolic array accelerators. Although they consider loop positioning into account, they is not a discussion about loop partitioning among different storage levels.

Orthogonal to the previous works, our work explores dataflows for 2-D spatial architectures for SNNs (Sec. 3.1) for rate coded spike representation. Additionally, we propose a set of directives to generate energy-optimal dataflows for a variety of workloads. Table 1 compares the contributions of different SNN hardware works.

3 SNN DATAFLOW OPTIMIZATION

3.1 Baseline Architecture and Dataflow

To represent a broader class of SNN and ANN spatial accelerators with multi-level storage and a two-dimensional array of PEs, we used an Eyeriss-like architecture as our baseline (SNN-Arch1) with modifications in the Processing Element (PE) for SNNs (Fig. 2). This baseline is similar to Eyeriss in the number of storage levels, buffer sizes and the size of the PE grid without restricting data movement in any dimension for a rigorous dataflow exploration. Each PE (Fig. 2(b)) has weight, input, Vmem and output buffers, threshold voltage (Vth) register, an accumulator, and a comparator unit. For the completeness of results and to observe the effect of changing hardware resources on optimal dataflow, we considered three variants of the hardware architecture. SNN-Arch2 has larger PE buffers, and SNN-Arch3 has larger PE buffers and more compute units in the PE than SNN-Arch1 (Table 2).

We selected an optimal ANN dataflow obtained from Timeloop [17] and added T as the outermost loop without changing the stationarity (for example, keeping WS ANN dataflow as WS SNN dataflow) to generate the baseline SNN dataflow for our analysis. Further, we identified the optimization opportunities in the baseline dataflow and generated a set of directives to utilize them.

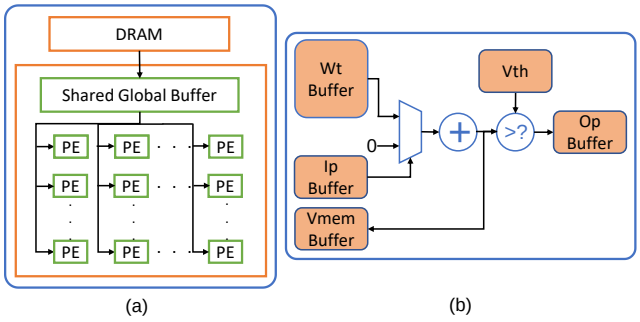

Figure 2: Eyeriss-like baseline SNN architecture (SNN-Arch1)

Table 2: Architecture Details

Component	SNN-Arch1	SNN-Arch2	SNN-Arch3
Description	Eyeriss-like	Eyeriss-like with more PE memory	Eyeriss-like with more PE memory and compute capacity
Weight Buffer	192x16 bit	128*128x16 bit	128*128x16 bit
Input Buffer	12x1 bit	128x1 bit	128x1 bit
Vmem Buffer	16x16 bit	128x16 bit	128x16 bit
Output Buffer	16x1 bit	128x1 bit	128x1 bit
Adder	1x16-bit	1x16-bit	128x16-bit
Comparator	1x16-bit	1x16-bit	128x16-bit

3.2 Factors that affect dataflows

Dataflow determines the execution of a workload on underlying hardware architecture. Two crucial factors affect the optimality of dataflows, namely, stationarity (data reuse) and utilization. We refer to an example OS dataflow for a dummy point-wise convolution layer (T=10, K=128, C=32, P=Q=14, N=R=S=1) mapped on a dummy hardware architecture similar to SNN-Arch1 but with four PEs, each having smaller buffers (scaled down by a factor of 4 for the ease of explanation) as shown in Fig. 3(ii) to discuss these factors.

- **Stationarity/Data-reuse**: Dataflow stationarity is defined by the most reused datastructure as explained in section 2.2. The example in Fig. 3(iii) represents an output stationary (OS) dataflow. The DRAM level loop (TKQC) has T, K and Q as outer dimensions but C as inner dimension. Consequently, to generate the tile of outputs corresponding to the T, K, and Q dimensions, the corresponding inputs, weights and Vmems will be fetched from and written back to the DRAM as needed. The datastructure whose indices do not include the inner dimensions is stationary. An optimal dataflow aims to maximize the reuse of different datastuctures.
- **Utilization**: On-chip resource utilization is another factor that helps in determining the optimality of any dataflow. Utilization corresponding to a dataflow refers to the percentage of total on-chip resources used for executing a workload on the underlying hardware architecture using that dataflow. On-chip resources include the number of PEs and the capacity of on-chip buffers. In Fig. 3(iii) baseline example, all the PEs are used but the buffers in each PE are underutilized. An optimal dataflow aims to maximize the utilization without

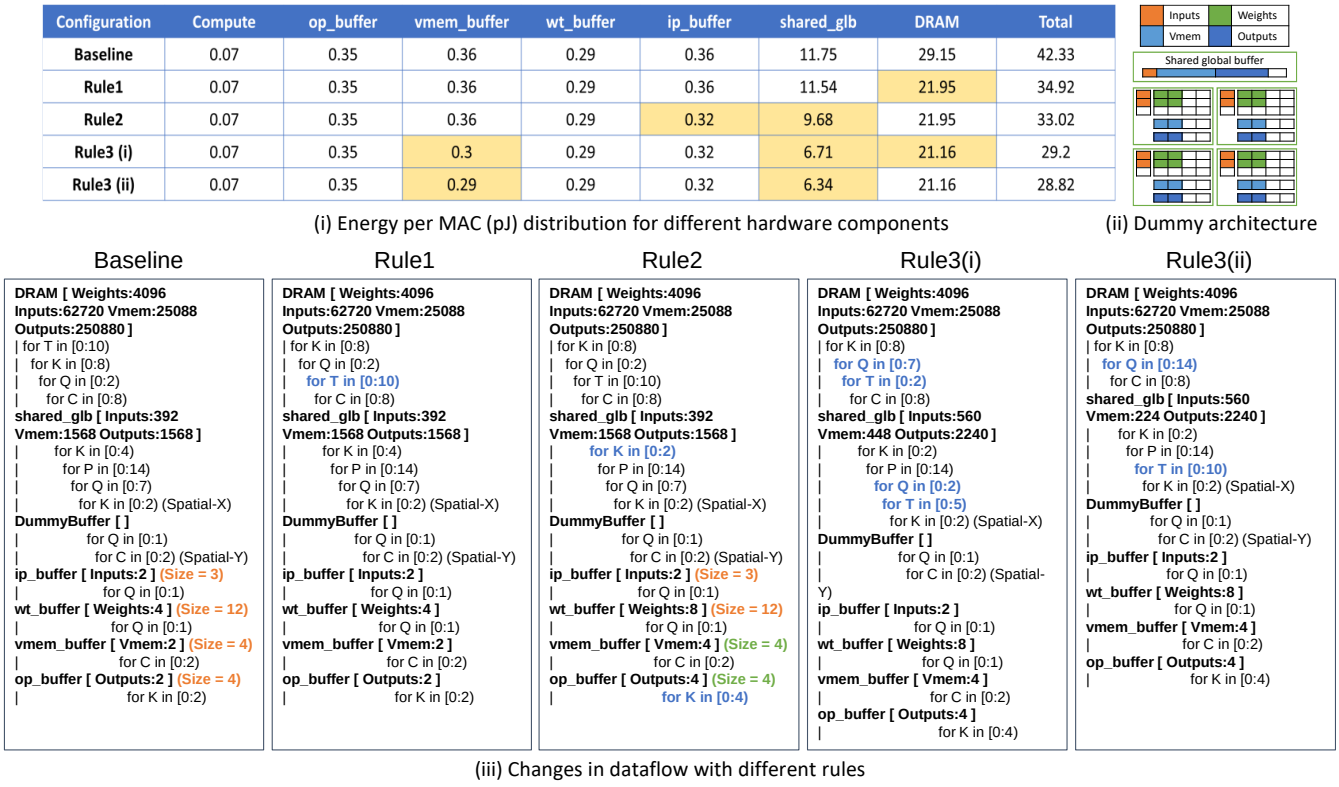

Figure 3: Output stationary (OS) dataflow example for dummy PWC layer

affecting the metric in consideration adversely (i.e. without increasing the EDP for this work).

3.3 SNN dataflow optimization rules

Changing the loop order changes the data reuse pattern, and changing the dimension partition among different storage levels changes the hardware utilization as well as data reuse. To improve the data reuse and hardware resource utilization in the baseline SNN dataflow (derived from an ANN dataflow), we propose three optimization directives/rules to perform systematic updates on any SNN dataflow making it more efficient. These rules increase data reuse and resource utilization by decreasing unnecessary data movement. We describe the optimization rules below and present their resultant improvements in section 5.

- **Rule 1 (T in)**: Making T as the innermost loop without changing the dataflow stationarity in any storage level decreases Vmem data movement and increases its data reuse. In other words, we calculate a block of outputs for all the timesteps before moving to the next (Fig. 3(ii)). For the example in figure Fig. 3, there is a 1.3x reduction in DRAM energy after applying Rule1.
- **Rule 2 (X in PE Buffer)**: Moving a factor of any dimension inside the PE buffer loop increases the buffer utilization without changing the actual number of PEs used to map the layer. For example, in Fig. 3(iii), a factor 2 of K is moved to op_buffer level from the global buffer level. It leads to a 1.17x reduction in global buffer energy and 1.13x reduction in ip_buffer energy by increasing utilization and data reuse.
- **Rule 3 (T in outer to inner level)**: Moving a factor of T dimension inside or to a storage level closer to compute unit, i.e. changing the partition/tiling in the T dimension decreases the energy of the lower storage level and improves data reuse. This rule may increase the total energy in some cases, and hence, should be applied with caution. For example, similar to the movement of K, moving T from DRAM to global buffer decreases the DRAM accesses (proportional to energy) but can increase the global buffer accesses. For comparable DRAM and global buffer energies, it may shift the bottleneck from DRAM to global buffer and increase the total energy. For the example in Fig. 3, this rule decreases the overall energy by increasing Vmem reuse and reducing the global buffer energy.

4 EXPERIMENTAL METHODOLOGY

We used Timeloop [17] for our analysis of different deep SNN dataflows. It is a tool for design space exploration of ANN Accelerators and contains two parts: a mapper and a model. Timeloop-model takes the problem (workload details), architecture and mapping descriptions as inputs and generates energy, latency and area statistics for computing that problem on the given hardware architecture and mapping. Similarly, Timeloop-mapper takes the problem, architecture and optional constraints as inputs and generates the most

Table 3: Workload Details

Name	Type	P/Q	R/S	K	C	T	N
Conv1/C1	VGG19 L9	28	3	512	256	100	1
Conv2/C2	VGG19 L13	14	3	512	512	100	1
Conv3/C3	VGG19 L4	112	3	128	128	100	1
DWC1/D1	Synthetic	56	3	NA	256	100	1
DWC2/D2	Synthetic	14	3	NA	512	100	1
DWC3/D3	Synthetic	224	3	NA	128	100	1
PWC1/P1	Synthetic	28	1	1024	256	100	1
PWC2/P2	Synthetic	14	1	2048	512	100	1
PWC3/P3	Synthetic	14	1	512	128	100	1

efficient mapping for the given problem and underlying architecture using the cost given by Timeloop-model. As Timeloop is a tool designed for ANNs, its cost model is optimized for ANNs. For this work, we did not modify the Timeloop-model. Instead, we added some correction factors to accommodate SNN specific details; for example, scaling the input read energy and output write for 1-bit spikes and considering only write accesses for output spikes. In addition, for a fair comparison and error minimization, we compare the EDP normalized to the baseline dataflow (Sec. 3.1) rather than the absolute numbers.

For our experiments, with the appropriate constraints for dataflow type, say row stationary and keeping T=1, we ran the Timeloop-mapper to get the most efficient ANN mapping using their random search option. Keeping T as the outermost loop and looking into the inefficiencies in this dataflow, we performed multiple experiments and formulated a set of rules as elaborated in section 3.3. Table 3 summarises the details of the workloads used for the experiments. To understand the dataflow dependence on different layer topologies, we consider the spiking versions of three types of popularly used convolution layers [9]: Regular (Conv), Depth-wise (DWC), and Point-wise Convolution (PWC) and, each with three configurations. For Conv layers, we chose one layer each from the initial, middle and later parts of the VGG19 network. For the DWC and PWC layers, we created synthetic benchmark layers with a varying range of dimensions. We observed similar trends in results for different timesteps therefore, in this paper, we only present results for 100 timesteps to compare the effects of other factors.

5 RESULTS

5.1 Effect of Rules on Dataflows

This section delineates the results of our experiments with different rules and how they affect energy and latency. The energy delay product (EDP) normalized to the baseline dataflows (3.1) is plotted in Fig. 4. Tables 2 and 3 contain details of the architectures and workloads, respectively. In Fig. 4, the rules are applied incrementally, i.e. Rule1 refers to Baseline with Rule1, Rule2 refers to Baseline with Rule1 and Rule2 and so on. Rule3 (i), (ii) and (iii) are the variants of Rule3 with increasing aggressiveness. To move T to an inner storage level i.e. Rule3, we may need to move another problem dimension to an outer storage level to meet on-chip memory constraints. In this context, a more aggressive version of Rule3 refers to a bigger

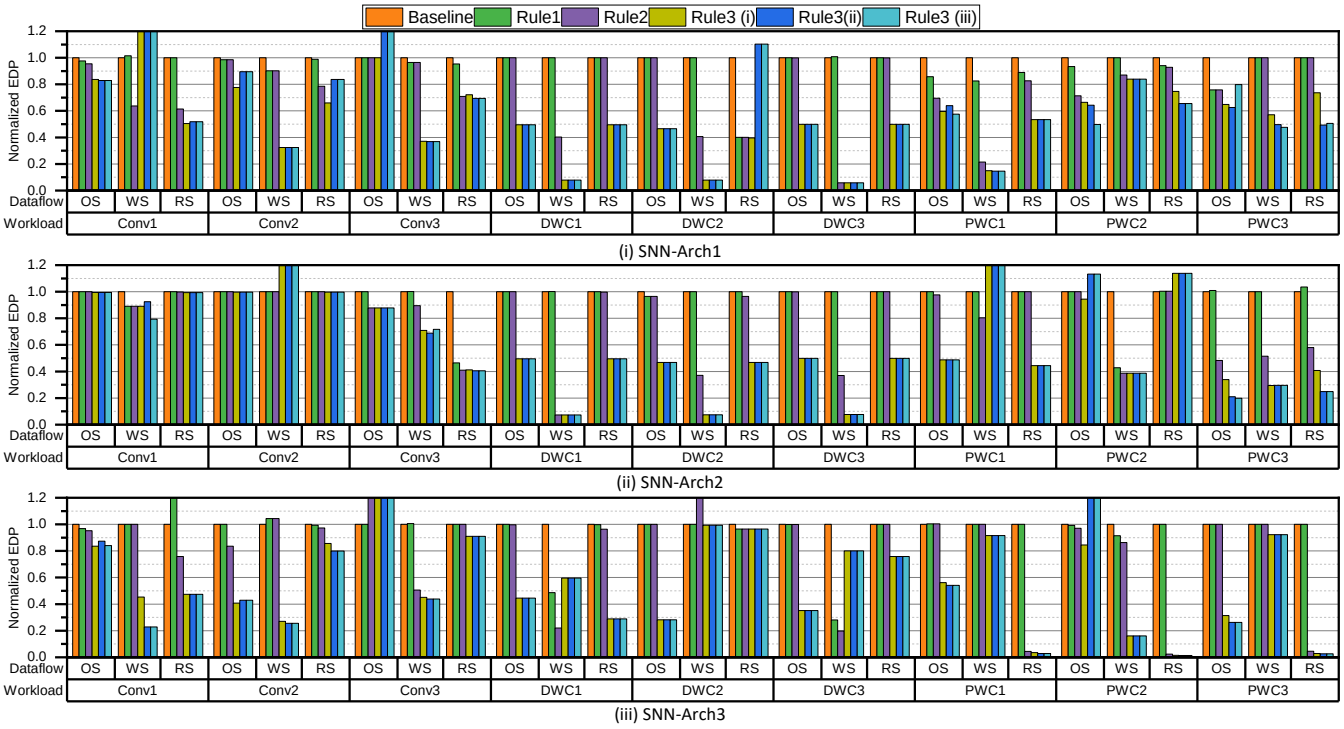

Figure 4: Comparison of EDP for different dataflow optimization rules normalized to the baseline for SNN-Arch1, 2 and 3.

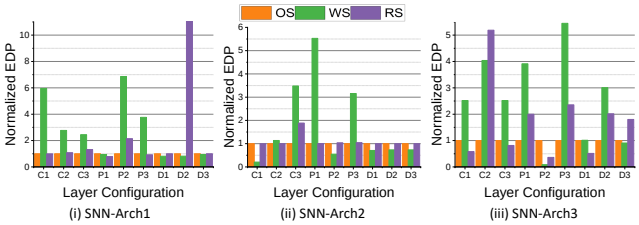

Figure 5: Comparison of OS, WS and RS dataflows for the three architectures and different layer configurations. The EDP is normailzed with respect to OS.

factor of T moved to an inner level. For example, in Fig. 3(ii)Rule3, to move T inside global buffer we changed the tiling factor of Q from 7 to 2.

For each plot in Fig 4, (i), (ii), and (iii) represent SNN-Arch1, 2 and 3, respectively. The x-axis refers to baseline dataflow and workload i.e., in Fig 4(i) RS PWC1 refers to row stationary dataflow for SNN-Arch1 and PWC layer configuration 1 (Table 3). The flat lines or no change in EDP in the graphs suggest that there is no opportunity to apply an optimization rule. For example, for SNN-Arch2, DWC3, baseline and Rule1 have same edp. It is because there is no way to move T without changing the stationarity. For Conv layers, the EDP reduces or remains the same (rule is not applicable) for a majority of dataflow and layer configurations when Rule1, Rule2 and Rule3(i) is applied. But for more aggressive versions of Rule3, there is not much reduction in EDP. It even increases in some cases. For PWC layers, the aggressive versions of Rule3 help in reducing the EDP in some cases. We can achieve up to 98% reduction in EDP for SNN-Arch3 RS dataflow. For DWC layers, any aggressive version of Rule3 does not change the EDP (or is not applicable) for most workloads and architectures and even increases for Architecture 1, RS dataflow for the DWC2 layer. Overall, Rule1 and Rule2 (when applicable) reduce the EDP for all the workloads in our experiments. Rule3 should be applied considering the available on-chip storage after applying Rule1 and Rule2.

5.2 Comparison of various dataflows

Fig. 5 compares the best performing versions (least EDP) of different dataflows for all workloads. There is no single dataflow that is optimal for all workloads and architectures. For example, for SNN-Arch3, PWC1 OS is optimal but, for PWC2, WS is the optimal dataflow. Whereas, for SNN-Arch1, PWC1 RS is optimal but, for PWC2, OS is the optimal dataflow. These results show that the optimal dataflow depends on the architecture as well as the workload and establishes the importance of SNN dataflow exploration.

6 CONCLUSIONS

Deep spiking neural networks can potentially perform certain machine learning tasks with high efficiency. While ANN hardware accelerators have gained popularity, there is still a need for improving SNN hardware efficiency. Dataflows play an important role in ANN hardware efficiency and ANN dataflow exploration has gained a lot of research interest. Whereas, such dataflows are relatively less explored in the context of SNNs. In this work, we analyzed dataflow inefficiencies for deep feed-forward SNNs and determined a set of directives for generating energy-efficient dataflows. We compare different dataflows and show that the optimal depends on the workload and the underlying architecture. Thus, we underscore the need for SNN dataflow exploration.

ACKNOWLEDGMENTS

The research was funded in part by the Center for Brain-Inspired Computing (C-BRIC), one of six centers in JUMP, a Semiconductor Research Corporation (SRC) program sponsored by DARPA, in part by the National Science Foundation (NSF), in part by IARPA MicroE4AI, and in part by the Department of Energy (DoE).

REFERENCES

[1] Akopyan et al. 2015. TrueNorth: Design and Tool Flow of a 65 mW 1 Million Neuron Programmable Neurosynaptic Chip. *IEEE TCAD* 34, 10 (2015). https://doi.org/10.1109/TCAD.2015.2474396
[2] Almomani et al. 2019. A comparative study on spiking neural network encoding schema: implemented with cloud computing. *Cluster Computing* 22, 2 (2019).
[3] Anthony N Burkitt. 2006. A review of the integrate-and-fire neuron model: I. Homogeneous synaptic input. *Biological cybernetics* 95, 1 (2006).
[4] Yongqiang Cao et al. 2015. Spiking deep convolutional neural networks for energy-efficient object recognition. *International Journal of Computer Vision* 113, 1 (2015).
[5] Chen et al. 2016. Eyeriss: A Spatial Architecture for Energy-Efficient Dataflow for Convolutional Neural Networks. In *ISCA*. https://doi.org/10.1109/ISCA.2016.40
[6] Chen et al. 2017. Using dataflow to optimize energy efficiency of deep neural network accelerators. *IEEE Micro* 37, 3 (2017).
[7] Davies et al. 2018. Loihi: A Neuromorphic Manycore Processor with On-Chip Learning. *IEEE Micro* 38, 1 (2018). https://doi.org/10.1109/MM.2018.112130359
[8] Diehl et al. 2015. Fast-classifying, high-accuracy spiking deep networks through weight and threshold balancing. In *IJCNN*. ieee.
[9] Howard et al. 2017. Mobilenets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861* (2017).
[10] Eugene M Izhikevich. 2004. Which model to use for cortical spiking neurons? *IEEE transactions on neural networks* 15, 5 (2004).
[11] Khan et al. 2008. SpiNNaker: mapping neural networks onto a massively-parallel chip multiprocessor. In *IJCNN*. IEEE.
[12] Lee et al. 2020. Enabling spike-based backpropagation for training deep neural network architectures. *Frontiers in neuroscience* 14 (2020).
[13] Lee et al. 2020. Spike-flownet: event-based optical flow estimation with energy-efficient hybrid neural networks. In *ECCV*. Springer.
[14] Jeong-Jun Lee et al. 2020. Reconfigurable dataflow optimization for spatiotemporal spiking neural computation on systolic array accelerators. In *2020 IEEE 38th International Conference on Computer Design (ICCD)*. IEEE.
[15] Li et al. 2016. Evaluating the energy efficiency of deep convolutional neural networks on CPUs and GPUs. In *BDCloud-SocialCom-SustainCom*. IEEE.
[16] Narayanan et al. 2020. Spinalflow: an architecture and dataflow tailored for spiking neural networks. In *ISCA*. IEEE.
[17] Parashar et al. 2019. Timeloop: A systematic approach to dnn accelerator evaluation. In *ISPASS*. IEEE.
[18] Rathi et al. 2020. DIET-SNN: Direct input encoding with leakage and threshold optimization in deep spiking neural networks. *arXiv preprint arXiv:2008.03658* (2020).
[19] Roy et al. 2017. A programmable event-driven architecture for evaluating spiking neural networks. In *ISLPED*. IEEE.
[20] Bodo Rueckauer et al. 2017. Conversion of continuous-valued deep networks to efficient event-driven networks for image classification. *Frontiers in neuroscience* 11 (2017).
[21] Sengupta et al. 2019. Going deeper in spiking neural networks: VGG and residual architectures. *Frontiers in neuroscience* 13 (2019).
[22] Shrestha et al. 2018. Slayer: Spike layer error reassignment in time. *arXiv preprint arXiv:1810.08646* (2018).
[23] Shihao Song et al. 2021. DFSynthesizer: Dataflow-based synthesis of spiking neural networks to neuromorphic hardware. *ACM Transactions on Embedded Computing Systems (TECS)* (2021).
[24] Wu et al. 2019. Direct training for spiking neural networks: Faster, larger, better. In *Proceedings of the AAAI*, Vol. 33.